Padrões de Projeto

App Inventra (Android, Kotlin + Jetpack Compose)

Ticket: INV2-152 · épico INV2-10
Repositório: InventraTech/inventra-mobile-development-2
Código analisado: commit `65bb923`
Autor: Felipe Kogake
Data: 25/10/2026

O padrão principal deste documento é o **Repository**. Ele não foi aplicado só para cumprir o requisito: é ele que permite testar as telas do app sem servidor. No fim há três padrões secundários que também aparecem no código.

1. Repository

Problema

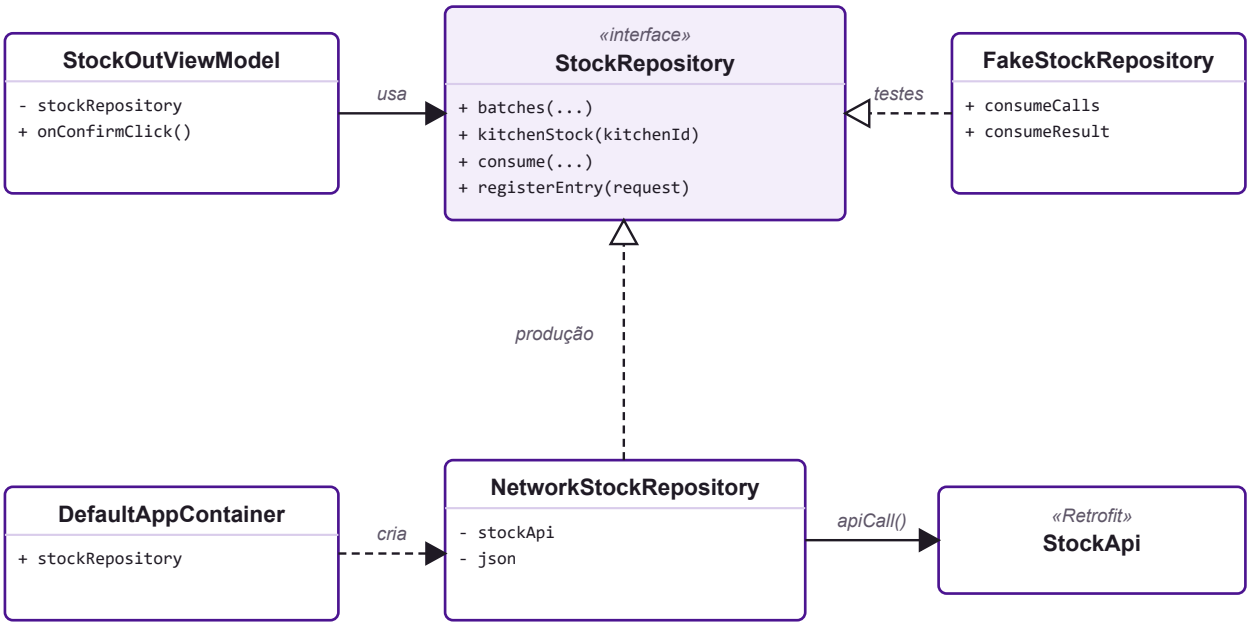
Os ViewModels precisam de dados do backend (estoque, requisições, fornecedores, sessão). Se chamassem o Retrofit direto, três coisas aconteceriam:

1. Para testar um ViewModel seria preciso subir um servidor ou simular respostas HTTP.
2. Tratar erros de rede, de HTTP e de JSON seria repetido em todo ViewModel.
3. Trocar a origem dos dados (API, cache, banco local) obrigaria a mexer em todas as telas.

Solução

Cada assunto do domínio tem uma interface `XxxRepository`, que diz **o que** pode ser feito. A classe `NetworkXxxRepository` diz **como**: chama a API e devolve `Result<T>`. O ViewModel só conhece a interface. Nos testes, a mesma interface é implementada por um `FakeXxxRepository`, que guarda as chamadas recebidas e devolve respostas prontas.

Diagrama de classes (exemplo da baixa de estoque)



Seta cheia: dependência. Seta tracejada com triângulo: implementa a interface. Seta tracejada simples: cria a instância.

Onde está no código

O exemplo é a baixa de estoque. O mesmo formato se repete nos 9 repositórios do app (Alert, Auth, Kitchen, KitchenAccess, Members, Product, Requisition, Stock e Supplier).

Papel	Arquivo	Detalhe
Contrato	StockRepository.kt	Linha 16: `interface StockRepository`
Implementação de produção	StockRepository.kt	Linha 49: `class NetworkStockRepository`
Tratamento único de erros	ApiCall.kt	`apiCall()` converte erros de HTTP, rede e JSON em `Result.failure` com mensagem em português
Criação da instância	AppContainer.kt	Linha 65: contrato. Linha 175: `NetworkStockRepository` criado com `lazy`
Consumo pela tela	StockOutViewModel.kt	Linhas 29 a 33: recebe `StockRepository` no construtor
Implementação de teste	FakeStockRepository.kt	Linha 18: classe. Linha 84: `consume` registra a chamada
Teste usando o fake	StockOutViewModelTest.kt	Linha 39: cria o fake. Linhas 122 e 134: validação sem chamar a API
Teste da implementação real	NetworkStockRepositoryTest.kt	Linha 22: testa o `NetworkStockRepository` contra uma API falsa

Como o padrão aparece num teste

O ViewModel valida a quantidade antes de dar baixa. Com o fake, o teste confere que a API **não** foi chamada, sem rede e sem servidor:

```
@Test
fun `quantidade maior que o saldo nao chama a API`() =
    runTest(testDispatcher) {
        val viewModel = createViewModel()
        advanceUntilIdle()
        viewModel.onQuantityChange("30")

        viewModel.onConfirmClick()

        assertEquals(
            "A quantidade passa do saldo da cozinha (24,5 kg)",
            viewModel.uiState.value.submitError,
        )
        assertTrue(stockRepository.consumeCalls.isEmpty())
    }
```

Resultado

- 9 repositórios no mesmo formato: interface, `Network*` e `Fake*`.
- 28 arquivos de teste de ViewModel e de navegação usam os fakes. Nenhum depende de rede.
- 9 testes de repositório verificam a implementação real contra APIs falsas.
- 431 testes unitários no total, todos rodando na JVM, sem emulador.
- Quando o backend muda um endpoint, só o `Network*Repository` e o DTO mudam. Os ViewModels e os testes deles continuam iguais.

2. Padrões secundários

Factory: criação dos ViewModels

Problema: o Android cria os ViewModels sozinho, então eles não podem receber dependências direto pelo construtor.

Onde: StockOutViewModel.kt, linhas 142 a 153. `factory(productId)` devolve um `ViewModelProvider.Factory` que busca os repositórios no `AppContainer`. A tela pede o ViewModel por essa fábrica em StockOutScreen.kt, linha 271. O mesmo formato se repete em todas as telas.

Chain of Responsibility: requisições HTTP

Problema: toda requisição precisa do token, e um token vencido precisa ser renovado sem que cada tela trate isso.

Onde: AppContainer.kt, linhas 120 a 126. A requisição passa pela cadeia do OkHttp nesta ordem:

1. AuthInterceptor (linha 15) põe o cabeçalho `Authorization: Bearer`.
2. O interceptor de log registra a chamada.
3. TokenAuthenticator (linha 23) entra só quando a resposta é 401: renova o token e repete a requisição.

Cada elo resolve uma parte e passa a requisição adiante.

Observer: estado das telas

Problema: a tela precisa ser redesenhada sempre que os dados mudam, sem o ViewModel conhecer a tela.

Onde: StockOutViewModel.kt, linhas 35 e 36, expõe `uiState: StateFlow<StockOutUiState>`. StockOutScreen.kt, linha 274, observa com `collectAsState()`. Cada `_uiState.update { ... }` notifica a tela, que se redesenha. Os testes leem o mesmo `uiState.value`, então não precisam de interface gráfica.